Manual Técnico — servicio-alquileres (Alquileres)

El microservicio **PrivateRentalService** forma parte del ecosistema de **TransportesGT**.

Su objetivo principal es coordinar y gestionar la reserva de buses privados en todo su flujo operativo:

- **Solicitud de alquiler:** realizada por el cliente desde la plataforma en línea o presencialmente en mostrador.
- **Estimación del costo:** calculada de forma automática aplicando las tarifas vigentes.
- **Cotización formal:** revisada y asignada por el administrador de la sucursal correspondientes.
- **Procesamiento del pago:** mediante la saldo de la cartera digital (orquestado por la saga con **Cartera**) o cobro directo en efectivo en mostrador.
- **Seguimiento del itinerario:** culmina con la creación de la entidad operativa de **Viajes**.

Especificaciones del Sistema

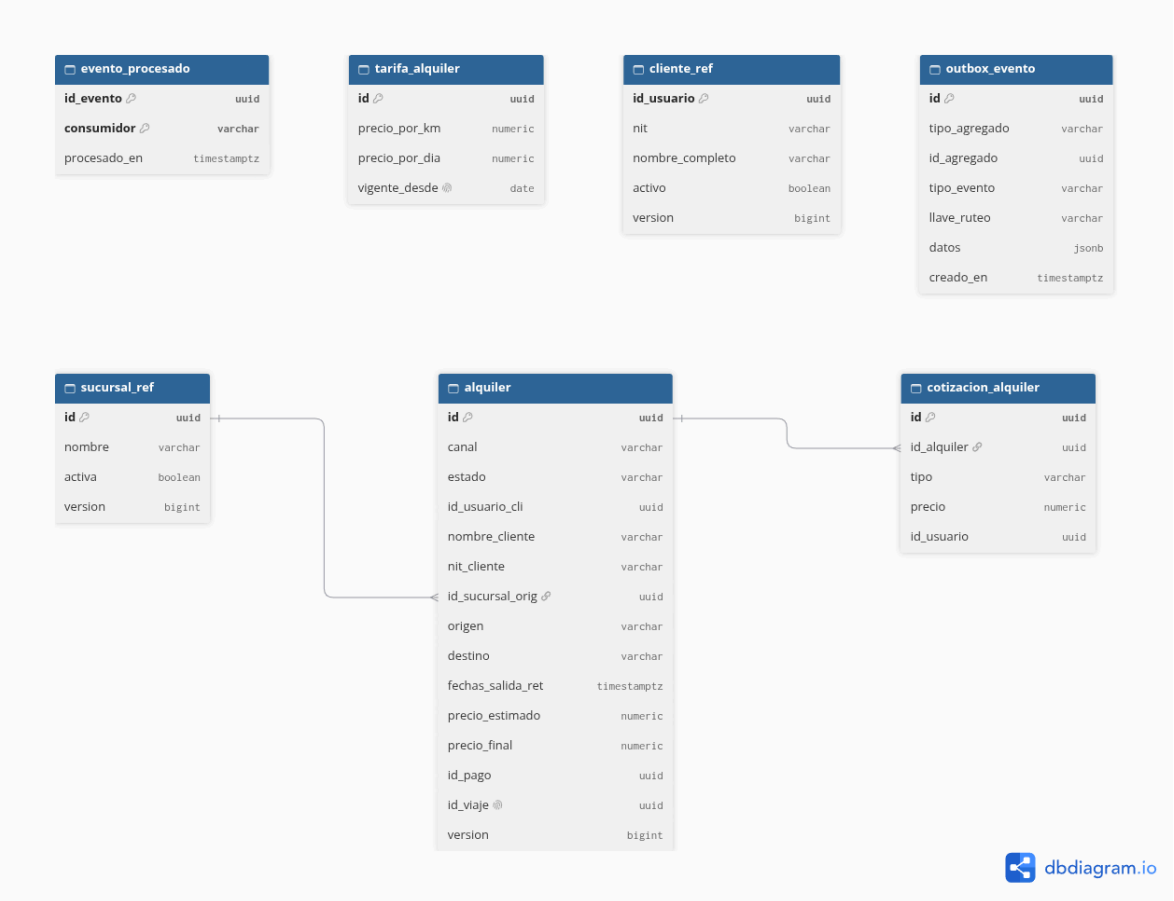
Parámetro	Detalle
Stack	Java 21 · Spring Boot 4.1.1 · Spring Data JPA (Hibernate) · Flyway · Spring AMQP · Spring Security (resource server JWT) · RestClient + Resilience4j · MapStruct · springdoc
Librería de eventos	com.github.luisbaquiax:EventosComunesTrasnportesGt:v2.1.1 (sobre, payloads, Nit, outbox, consumidor idempotente, topología)
Puerto / ruta base	8086 / /v1/rentals
Base de datos	PostgreSQL bd_alquileres (usuario svc_alquileres)

Parámetro	Detalle
Broker	RabbitMQ, exchange topic transporte.eventos, DLX transporte.eventos.dlx

Migraciones Flyway

Migración	Contenido
V1__esquema_inicial.sql	Tablas, restricciones y trigger de transiciones
V2__datos_prueba.sql	Proyecciones de sucursales y perfiles, tarifa inicial y un alquiler en cada estado (datos-semilla.md)

1. Arquitectura de Datos y Tablas



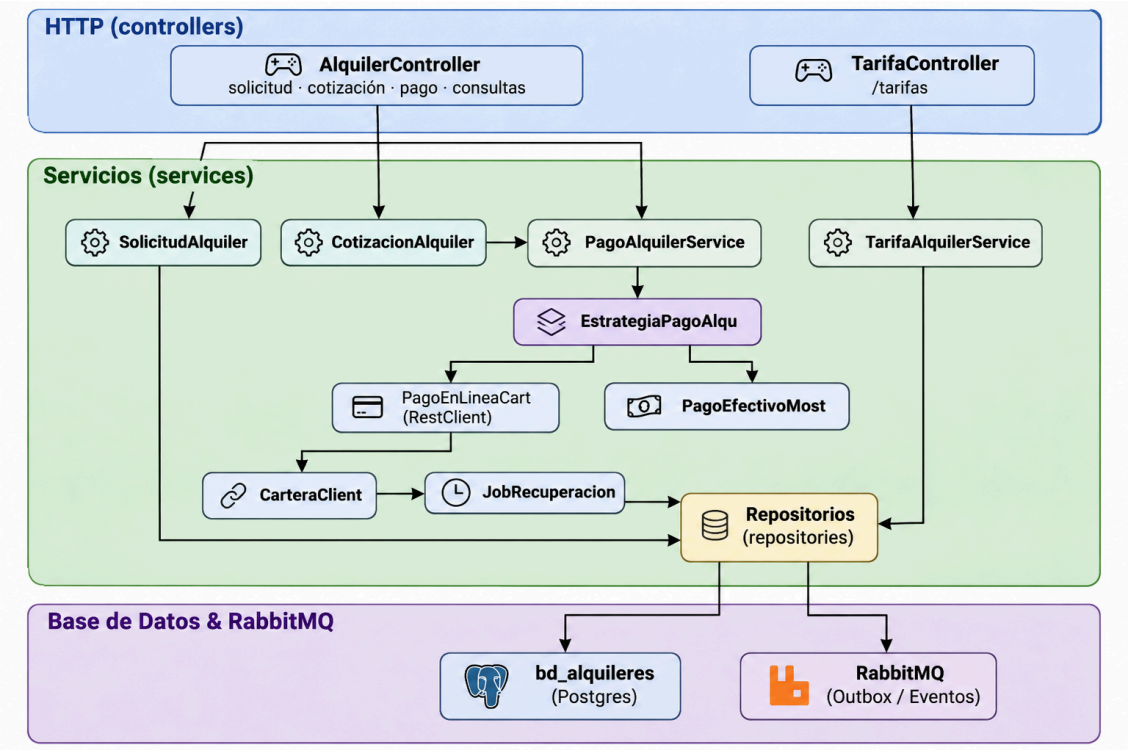
Nota: Las proyecciones sucursal_ref y cliente_ref se actualizan mediante la sincronización de eventos de dominio y no dependen de llaves foráneas (FK), a excepción del origen de la sucursal. La búsqueda en cliente_ref se efectúa directamente a través del id_usuario.

Restricciones de Base de Datos

Restricción	Garantiza
trg_alquiler_transicion (fn_validar_transicion_estado)	Solo las transiciones SOLICITADO>COTIZADO, SOLICITADO>CANCELADO, COTIZADO>CONFIRMADO, COTIZADO>CANCELADO, CONFIRMADO>FINALIZADO y CONFIRMADO>CANCELADO; el mismo estado (reajustar una cotización) no se valida.
ck_alquiler_coordenadas	Las cuatro coordenadas vienen juntas o ninguna.
ck_alquiler_retorno	Ida y vuelta exige un retorno posterior a la salida; solo ida no lo admite.
ck_alquiler_confirmado	CONFIRMADO/FINALIZADO tienen precio final, fecha de pago y llegada estimada, y en línea también id_pago.
ck_alquiler_llegada	La llegada estimada es posterior a la salida y, en ida y vuelta, no anterior al retorno.
uq_alquiler_viaje (parcial)	Un viaje por alquiler.
tarifa_alquiler.vigente_desde UNIQUE	Una tarifa por fecha de vigencia (registrar otra con la misma fecha la reemplaza).

*El servicio valida estas reglas de negocio en la capa de aplicación antes de persistir los datos, retornando respuestas HTTP **400**, **409** o **422** con detalles claros sobre el error. La base de datos actúa como una capa final de resguardo.*

2. Diagrama de Componentes y Capas



Organización de Capas y Paquetes

Capa	Paquete	Responsabilidad
Web	controllers	Contrato HTTP, @Valid, paginación pagina/tamano (@Min/@Max, máx. 100), Location en los 201. Solo dependen de interfaces.
Seguridad	config.security	JWT, reglas gruesas por rol (SecurityConfig), ConvertidorUsuarioAutenticado (claims → UsuarioAutenticado) y respuestas 401/403 en JSON.
Negocio	services.<tema> con su impl	Una interfaz por caso de uso.• **solicitud**: validación, distancia (Haversine × factor

Capa	Paquete	Responsabilidad
		de carretera) y precio estimado.• **cotizacion**: precio final y llegada.• **pago**: flujo común y Strategy por canal.• **consulta**: detalle, listado e historial.• **tarifa**: vigente, historial y registro.• **proyeccion**: réplicas _ref y seguimiento del viaje.
Reglas compartidas	services.comun, services.autorizacion	TransicionesAlquiler (mismo mapa que el trigger), GuardadoAlquiler (flush antes de publicar), NormalizadorNit (envuelve Nit de la librería), CalendarioGuatemala, AutorizacionAlquiler (gestión por sucursal, dueño, lectura por NIT).
Integración	clients	CarteraClient: 4xx → CarteraRechazoException con el status y código de Cartera; red, 5xx o circuito abierto → ServicioNoDisponibleException (503).
Eventos	eventos	Manejadores ManejadorEvento<T> que solo delegan, y EventosAlquiler, que arma los payloads con MapStruct.
Job	jobs	Recuperación de pagos en línea que quedaron en duda.

Capa	Paquete	Responsabilidad
Datos	models, repositories	Entidades (ddl-auto=validate), AlquilerSpecs (filtros y visibilidad en la base) y upsert nativos de las proyecciones.
Errores	exception	Jerarquía AlquilerException, FabricaRespuestaError y GlobalAlquilerExceptionHandler.

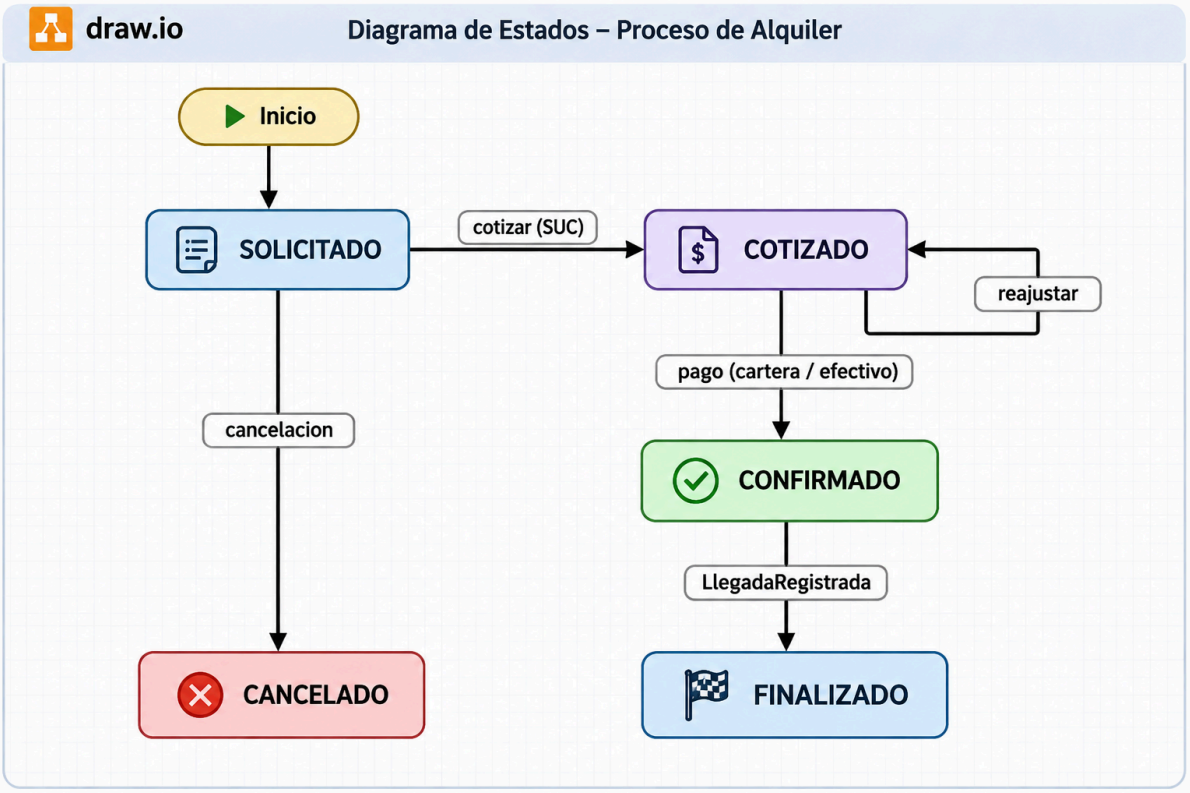
Nota arquitectónica: Se evita el uso de métodos estáticos en la lógica del servicio; las clases auxiliares y utilitarios están definidos como componentes inyectables (@Component).

2.1 Endpoints y Permisos

Método	Ruta	Quién	Resultado
POST	/alquileres/solicitudes	Autenticado con perfil y…	201 SOLICITADO + AlquilerSolicitado · 422 PERFIL_INCOMPLETO, COORDENADAS_*, SIN_TARIFA
POST	/alquileres/mostrador	SUC, CAJ	201. Cuerpo anidado { solicitud, nitCliente?, nombreCliente }; 400 NIT_INVALIDO
POST	/alquileres/{id}/cotizacion	SUC de la sucursal	200 COTIZADO + AlquilerCotizado · 409 TRANSICION_INVALIDA
POST	/alquileres/{id}/pago	El cliente dueño	200 CONFIRMADO + AlquilerConfirmado · 4xx de Cartera · 503

Método	Ruta	Quién	Resultado
POST	/alquileres/{id}/pago-mostrador	SUC, CAJ de la sucursal	200 CONFIRMADO (efectivo, idPago = null)
GET	/alquileres/mios	Autenticado	Suyos en línea y de mostrador con el NIT de su perfil
GET	/alquileres?estado&desde&hasta&sinAsignar	SUC, CAJ (su sucursal)…	Días de Guatemala incluidos; sinAsignar = CONFIRMADO sin bus
GET	/alquileres/{id}	SIS, personal de la suc…	Detalle con historial de cotizaciones
GET	/alquileres/tarifas/vigente?fecha	Autenticado	404 SIN_TARIFA
GET / POST	/alquileres/tarifas	SIS	Historial / registrar (reemplaza la de la misma fecha; no admite fechas pasadas)

3. Ciclo de Vida y Precio Estimado



Cálculo del Precio Estimado

Fórmula de Cálculo:
distancia × precioPorKm × (2 si es ida y vuelta) + días × precioPorDia
El resultado se redondea a 2 decimales mediante el método **HALF_UP** y se calcula tomando como referencia la tarifa vigente a la fecha en Guatemala.

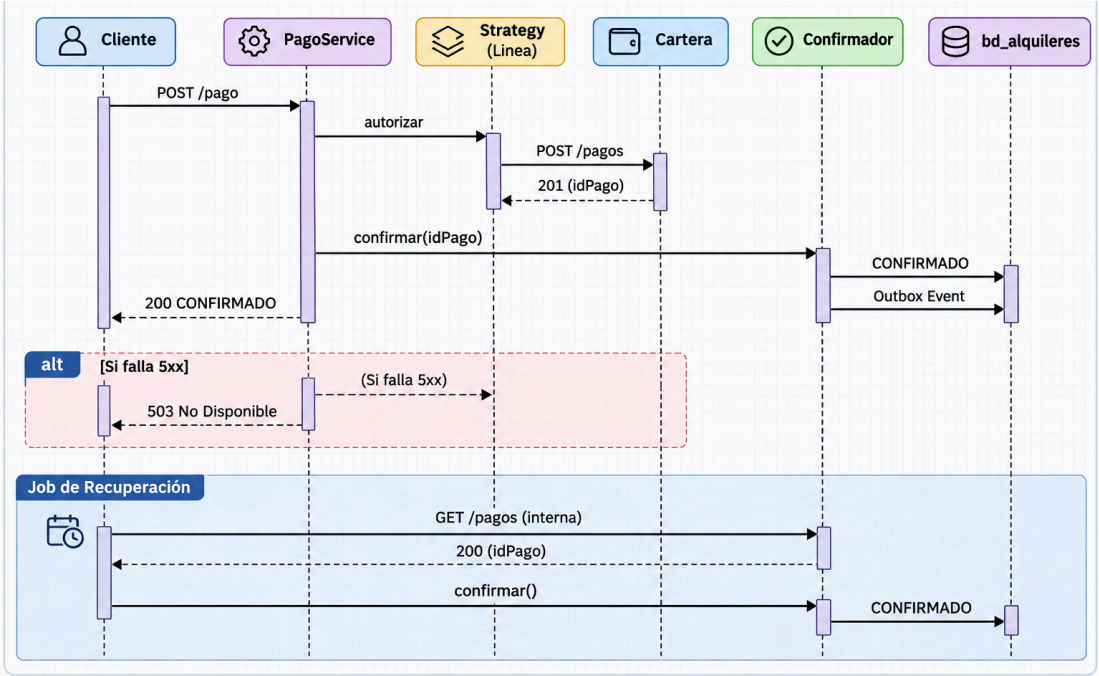
- **Cálculo de Distancia:** Se evalúa la distancia Haversine entre las coordenadas provistas por la propiedad alquileres.factor-carretera (valor predeterminado: **1.3**). En ausencia de coordenadas precisas, se toma el valor especificado en distanciaEstimadaKm.
- **Duración en Días:** Comprende los días de calendario (hora oficial de Guatemala) desde la salida hasta el retorno inclusive. Los viajes marcados solo como ida equivalen a **1 día** de servicio.

El precio estimado resultante se registra en la tabla cotizacion_alquiler bajo el tipo **ESTIMADA**. Cualquier modificación o ajuste posterior realizado por el administrador de la sucursal quedará guardado con el tipo **AJUSTADA_ADMIN**.

4. Saga de Pago en Línea y Job de Recuperación

El componente **PagoAlquilerService** ejecuta el flujo de procesamiento de pago de manera unificada para ambos canales. La estrategia de canal específica determina el proceso de verificación y cobro:

1. Carga el alquiler.
2. La estrategia autoriza.
3. Comprueba que el canal del alquiler sea el del endpoint.
4. Verifica la transición a CONFIRMADO.
5. La estrategia cobra.
6. ConfirmadorAlquiler confirma.



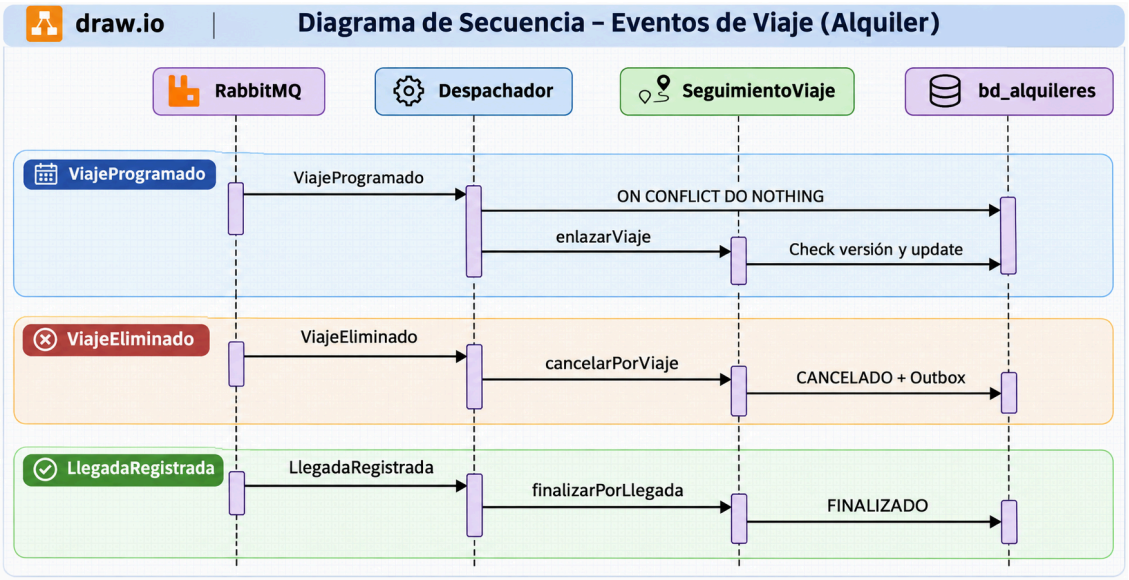
Reglas Clave de la Saga

- **Manejo de fallos en pago en línea:** Se asegura la consistencia impidiendo la confirmación de alquileres en línea sin un identificador válido (idPago). Esto se complementa con la restricción ck_alquiler_confirmado a nivel de base de datos.
- **Garantía de idempotencia:** El servicio de Cartera evita cargos duplicados para una misma reserva asociando las transacciones al idAlquiler. Esto garantiza la reintento seguro desde el cliente ante un error HTTP 503.
- **Secuencia de versión del evento:** El componente GuardadoAlquiler efectúa una llamada a saveAndFlush previo a la emisión del evento. Esto garantiza

que versionAgregado mantenga el valor actualizado, previniendo que los consumidores omitan eventos legítimos.

- **Validación de pagos en mostrador:** La clase PagoEfectivoMostrador autoriza el cobro verificando únicamente credenciales de usuario con rol SUC o CAJ pertenecientes a la sucursal del alquiler, omitiendo las llamadas a Cartera.

5. Eventos Consumidos y Publicados



Catálogo de Eventos

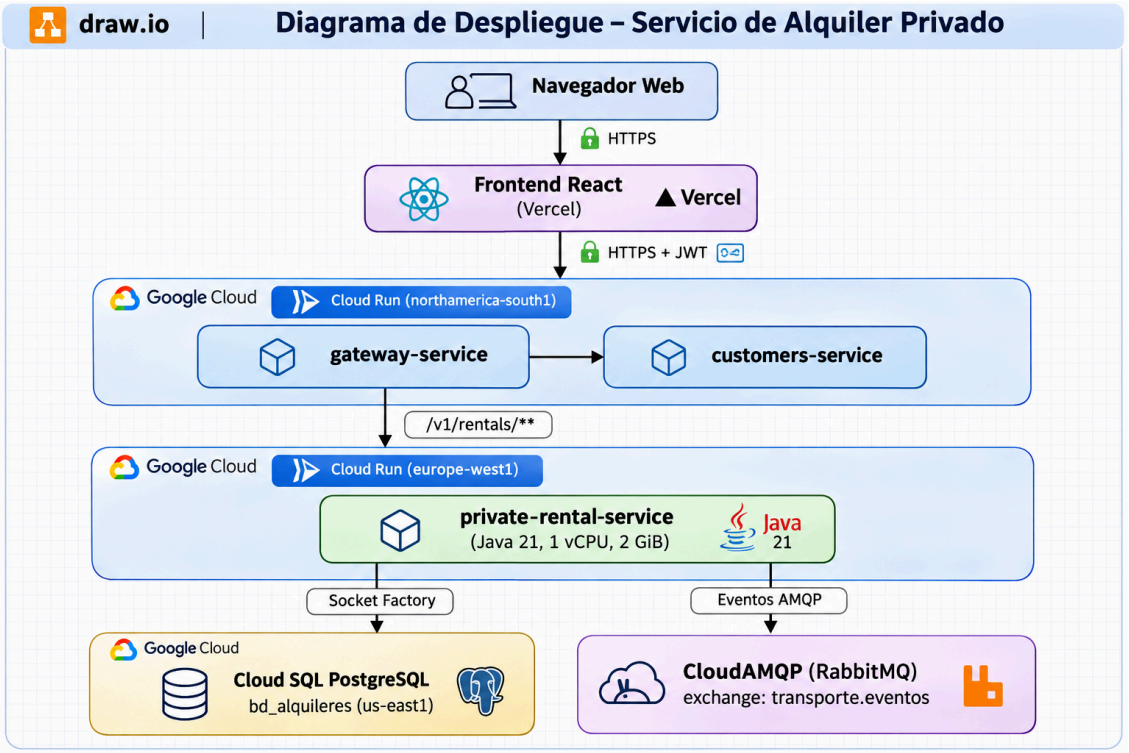
Evento	Cola	Efecto
SucursalCreada / SucursalActualizada	alquileres.identidad.q	*Upsert* de sucursal_ref por versión.
PerfilClienteActualizado	alquileres.cartera.q	*Upsert* de cliente_ref con el NIT normalizado (habilita la solicitud en línea y el historial por NIT).
ViajePrograma / ViajeActualizado	alquileres.viajes.q	Enlaza id_viaje y refleja bus y chofer; ignora las versiones viejas.
ViajeEliminado	alquileres.viajes.q	Cancela y publica AlquilerCancelado.

Evento	Cola	Efecto
LlegadaRegistrada	alquileres.viajes.q	Finaliza el alquiler.

Evento Publicado	Llave	Consumidores
AlquilerSolicitado	alquileres.alquiler.solicitado	Notificaciones
AlquilerCotizado	alquileres.alquiler.cotizado	Notificaciones
AlquilerConfirmado	alquileres.alquiler.confirmado	Viajes (crea el viaje), R…
AlquilerCancelado	alquileres.alquiler.cancelado	Clientes y Cartera (ree…

Manejo de reintentos y errores transitorios: Ante fallas temporales, se realizan hasta 4 reintentos de procesamiento (con intervalos de 1, 2, 4 y 8 segundos, tope máximo 10 s). Si persisten los errores o se detecta un payload JSON inválido, el mensaje se redirige directamente a la cola de fallos (.dlq).*

6. Diagrama de Despliegue



Especificaciones de Infraestructura y Despliegue

Elemento	Detalle
Imagen	Dockerfile en dos etapas: maven:3.9-eclipse-temurin-21 compila (./mvnw clean package) y bellsoft/liberica-openjdk-alpine:21 ejecuta app.jar.
CI/CD	Cada merge a main dispara el activador de Cloud Build: docker build PrivateRentalService -f PrivateRentalService/Dockerfile, push a Artifact Registry con la etiqueta del commit y gcloud run services update private-rental-service.
Base de datos	Cloud SQL unimarketcunoc:us-east1:transportesgtdb, base bd_alquileres, por la *socket factory* (jdbc:postgresql:///bd_alquileres?cloudSqlInstance=…&socketFactory=…).
Configuración	Variables de entorno: SPRING_PROFILES_DEFAULT=prod, SPRING_DATASOURCE_*, RABBITMQ_URL, JWT_SECRET (el mismo de todos los servicios), CARTERA_URL, CLAVE_INTERNA, ALQUILERES_FACTOR_CARRETERA, ALQUILERES_RECUPERACION_INTERVALO_MS, SERVER_PORT=8080, SWAGGER_HABILITADO, DB_POOL_MAX.
Esquema	Flyway aplica las migraciones al arrancar; Hibernate solo valida. En la nube no se modifican migraciones ya aplicadas: los cambios van en una versión nueva.

Elemento	Detalle
Instancias	Exactamente 1 (min y max). Varias serían seguras: el outbox y evento_procesado hacen idempotente el consumo.
CPU	Hoy con CPU solo durante peticiones (valor por defecto). Sus jobs (@Scheduled) y los consumidores de RabbitMQ necesitan CPU sin tráfico HTTP: conviene --no-cpu-throttling.
Región	Es el único servicio en **europe-west1**; llega a Cloud SQL (us-east1) y a Cartera (northamerica-south1) por internet, igual que los demás.
Llamadas	Cobra el pago en línea en Cartera; el job de recuperación consulta GET /cartera/pagos con X-Clave-Interna.
Acceso	Público (ingress=all, sin IAM de invocación); los datos los protege el JWT. Pendiente: dejar público solo el gateway.
Cuenta de servicio	La de Compute por defecto (…-compute@developer.gserviceaccount.com), compartida con los demás servicios.
Salud	/v1/rentals/actuator/health (liveness y readiness).